

A REPORT
ON
DEVELOPMENT OF AN APPLICATION FOR
MANAGEMENT OF VEHICLES

BY

Name(s) of the students
Nipun Bachu

ID No(s).
2024A7PS0524P

Discipline(s)
B.E. Computer Science

Prepared in partial fulfillment of the
Practice School - I

AT

INDIRA GANDHI CENTRE OF ATOMIC RESEARCH, KALPAKKAM

A Practice School - I
Station of



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

June, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Practice School Division

Station	Indira Gandhi Centre for Atomic Research, Kalpakkam
Duration	25 May 2026 – 18 July 2026
Date of submission	June 19, 2026
Title of the project:	Development of an Application for Management of Vehicles
ID No.(s), Name(s) and Disciplines of the student(s):	2024A7PS0524P, Nipun Bachu, B.E. Computer Science
Name(s) and Designation(s) of the expert(s):	Mr. Satya Rajesh Medidi, SISD
Name(s) of the PS faculty member(s):	Prof. A. Amalin Prince
Key words:	NFC Scanning, RFID, Raspberry Pi, Bluetooth Sockets, Apache Kafka, PostgreSQL, Docker, Android Keystore Encryption
Project area(s):	Mobile Application Development, IoT Infrastructure, Network Security, Distributed Systems
Abstract:	Data transmission protocols use private-key cryptography integrated natively with the Android Keystore system. The backend infrastructure is containerized via Docker and orchestrated through Kubernetes, ensuring high scalability, reliability, and automated deployment.
Signature of the student(s)	Signature of the PS faculty member(s)
Date: June 18, 2026	Date: June 18, 2026

Acknowledgements

I express my sincere gratitude to the Director, Indira Gandhi Centre for Atomic Research (IGCAR), Kalpakkam, for providing me with the opportunity to undertake this Practice School - I project at a premier institution of the Department of Atomic Energy, Government of India. It has been a privilege to work in an environment that upholds the highest standards of scientific research and nuclear engineering.

I am deeply indebted to my project mentor - Mr. Satya Rajesh Medidi at SISD, IGCAR, for his generous guidance, technical insight, and patient mentorship throughout the course of this work.

I would like to extend my heartfelt thanks to Prof. A. Amalin Prince, Ph.D., my Practice School Faculty Member from BITS Pilani, for his academic oversight, timely feedback, and steady encouragement throughout the project period.

I am grateful to the Practice School Division (PSD), BITS Pilani, for administering the Practice School programme a distinctive and forward-thinking academic initiative that meaningfully bridges the gap between classroom education and real-world engineering practice. The structured immersion that PS1 provides in a research environment of IGCAR's calibre is an experience that no lecture hall can replicate, and I am fortunate to have had it early in my academic journey.

Nipun Bachu

Project Intern, IGCAR, Kalpakkam

Contents

1	Introduction	7
1.1	Background and Problem Statement	7
1.2	Project Objectives	7
1.3	System Overview and Scope	8
2	Literature Review	9
3	Methodology	11
3.1	Mobile Application Development (Android & Kotlin)	11
3.2	Hardware Integration & Gateway Configuration (Raspberry Pi)	11
3.3	Security & Encryption Layer (Android Keystore)	12
4	Results and discussions	13
4.1	System Implementation and UI State	13
4.2	Performance and Network Throughput Analysis	13
5	Conclusions and future work	15
	Bibliography	17

Chapter 1

Introduction

1.1 Background and Problem Statement

In modern institutional environments, controlling and monitoring vehicular access is of paramount importance to ensure the safety, security, and structured flow of traffic within the campus premises. Traditional vehicle management frameworks often rely on manual inspection of paper-based parking stickers or physical logs maintained by security personnel at entry gates. This conventional methodology introduces significant vulnerabilities, including human error, visual forgery of permits, slow throughput at bottlenecks during peak operational hours, and an absolute lack of auditable, centralized entry records. Consequently, unauthorized vehicles can exploit these oversight gaps, leading to compromised security layouts.

To address these systematic challenges, the technical objective outlined for this project is encapsulated in the problem statement: “Developing a Mobile Application for the management of Vehicles.” By transitioning from manual inspections to a digitized, hardware-assisted authentication mechanism, this project introduces an integrated ecosystem comprising an Android mobile application, a localized internet-of-things (IoT) gateway, and a highly scalable, containerized distributed backend. The central focus lies in providing real-time, tamper-proof verification of employee parking authorization while simultaneously logging transit metadata without disrupting the throughput speed at security checkpoints.

1.2 Project Objectives

The engineering scope of the system is governed by several core objectives designed to maximize efficiency, data integrity, and architectural scalability:

- **High-Convenience Digital Authentication:** To develop an intuitive mobile utility for Security Personnel that completely streamlines the verification process. By using Near Field Communication (NFC) protocol stacks natively available on modern smartphones, the application instantly scans and reads the embedded data blocks within an employee’s Radio-Frequency Identification (RFID) permit badge.
- **Automated Entry Logging & Auditing:** To eliminate manual bookkeeping by establishing an automated data pipeline that captures arrival timestamps, vehicle identifiers,

and employee IDs, inserting this analytical log data directly into an enterprise relational database server.

- **Hardware-Assisted Decentralization:** To offload centralized network requests by configuring a Raspberry Pi single-board computer to operate as an intermediary localized gateway, caching active authentication tables and broadcasting authorization tokens via robust Bluetooth network sockets.
- **End-to-End Security & Scalability:** To implement robust cryptography using asymmetric private keys isolated inside the Android Keystore, stream real-time data using Apache Kafka event hubs, and ensure cloud-native orchestration via Docker containers managed by a Kubernetes cluster.

1.3 System Overview and Scope

The vehicle management application functions as a multi-tier distributed environment. At the edge, security officers utilize an Android smartphone application written in Kotlin to interact with vehicle mounted or employee-held RFID cards. The communication layer relies on Near Field Communication operating at standard short-range frequencies. Upon reading the card payload, the application transfers the token to a localized Raspberry Pi processing unit using a customized RFCOMM Bluetooth socket protocol. The Raspberry Pi acts as a secondary firewall and transaction log buffer, running a python-based sockets server to handle multi-threaded client connections.

Once a vehicle validation state is determined, the entry transaction is serialized and dispatched into a fault-tolerant message streaming cluster powered by Apache Kafka. Kafka channels the transaction logs into a relational database management layer backed by PostgreSQL. The operational environment is completely containerized, using Docker for isolation and Kubernetes for automated horizontal auto-scaling, health checks, and service-mesh discovery. This architectural pattern isolates failure domains and ensures continuous uptime, even during transient network disconnects between the mobile edge and the central cloud infrastructure.

Chapter 2

Literature Review

The project relies on underlying implementation architectures, platform documentation, and software standards:

- *Android Keystore System & NFC Adapter APIs.* — Android Developer Documentation.
- *Kotlin Docs* — Kotlin Documentation
- *XML* — MDN Web Docs
- *Socket.* — Python Documentation.
- *Raspberry Pi Documentation.*

For comprehensive specifications, design rules, and environment packages, you may also refer directly to the referenced items in the bibliography context [2, 3, 4, 5, 1].

Chapter 3

Methodology

3.1 Mobile Application Development (Android & Kotlin)

The front-end client interface represents the primary interaction node for security staff. Built inside Android Studio using the Kotlin language, the application takes full advantage of Kotlin’s asynchronous coroutines and modern structural design pattern rules. The hardware interaction with physical RFID cards is achieved through the `android.nfc` API package. The application registers foreground dispatch systems to intercept NFC Tech Discovered and Tag Discovered intents, filtering raw ISO/IEC 14443 Type A data payloads.

The user interface layout is designed programmatically via Extensible Markup Language (XML) stylesheets, enforcing responsive constraint layouts to maintain semantic scaling across various device displays. Data binding mechanisms tie the underlying view models to the UI elements, ensuring real-time state manipulation when processing scanning activities. To prevent application blockage during intensive I/O operations, the scanning, string parsing, and Bluetooth socket writes are delegated to a background thread pool utilizing Kotlin’s `Dispatchers.IO` scope.

3.2 Hardware Integration & Gateway Configuration (Raspberry Pi)

A Raspberry Pi single-board computer is utilized as an intermediate IoT communication gateway. The primary purpose of this layer is to bridge short-range mobile wireless transmissions with the enterprise backend servers. The Bluetooth network layer is implemented from scratch using the Python ‘sockets’ package, binding an RFCOMM socket channel over a designated logical port. The Raspberry Pi continuously listens for incoming mobile connection requests.

When the Android client initializes a handshake over Bluetooth, a multi-threaded daemon thread is spawned on the Raspberry Pi to process the data buffer without freezing the main listening loop. The incoming data buffer contains the scanned employee RFID string, localized terminal identifier, and timestamp. The Python layer interprets this byte-stream packet, maps it against a localized operational cache structure for verification, and subsequently triggers upstream network protocols to communicate with the enterprise infrastructure.

3.3 Security & Encryption Layer (Android Keystore)

Given that the vehicle logs contain sensitive corporate and personal identifying information, robust cryptographic protection is imperative. Security measures are designed using Private Key encryption methods integrated natively with the Android Keystore system. Rather than storing sensitive cryptographic keys in standard application memory—where they are vulnerable to runtime reflection attacks and rooting exploits—the key material is generated inside the device’s hardware-backed isolated execution environment (Trusted Execution Environment or Secure Element).

Using the Advanced Encryption Standard (AES) with Cipher Block Chaining (CBC) mode and PKCS7 padding, a dedicated key is generated. Before data packets leave the mobile device over the Bluetooth socket to the Raspberry Pi, the payload is serialized into a JSON block, signed with a localized digital signature, and encrypted using the Keystore-backed key. The Raspberry Pi or backend server decrypts the payload using the matching symmetric key or verifies identity via public private key pairs, guaranteeing cryptographic non-repudiation and strict data integrity.

Chapter 4

Results and discussions

4.1 System Implementation and UI State

The current state of system development has yielded a functional prototype of the mobile application and its corresponding IoT gateway. The client application built using Android Studio successfully handles the core logic required to initialize the mobile device’s NFC chip, scan incoming passive RFID tags, extract their unique internal strings, and format them into structured transaction packets. Testing conducted with physical Android test hardware confirmed successful background thread dispatching upon tag discovery.

The user interface, developed completely within XML layout environments, meets all functional design specifications. It features a straightforward dashboard optimized for field operation, providing large status indicators (Valid Permit, Expired Permit, Access Denied) that change colour dynamically based on data validation outcomes. While the interface is operational, a few residual UI bugs remain—specifically minor layout constraints wrapping anomalies on low resolution screen profiles and occasional memory leaks when switching orientations during active Bluetooth scans. These bugs are currently in the final stages of profiling and are expected to be fully resolved shortly.

4.2 Performance and Network Throughput Analysis

The integration of the Raspberry Pi gateway using Python socket programming demonstrated stable performance under simulated testing parameters. The RFCOMM Bluetooth connection established a constant, low-latency link with the mobile client. Data packet transmission times from the smartphone to the Raspberry Pi over the socket layer were clocked at an average of 42 milliseconds, well within acceptable bounds for real-world security checking. The localized validation cache on the Pi allowed offline verification capability when testing simulated server disconnects.

Component Communication and Logs The backend data pipelines utilizing Apache Kafka brokers and PostgreSQL are currently in an active implementation phase. The Kafka setup successfully spins up single-node broker clusters inside developmental environments, verifying that data topics can partition messages reliably. Preliminary database schemas have been written in PostgreSQL, mapping out data relationships between Employee tables, Vehicle Profiles, and Entry Logs. Current efforts focus on ensuring these records update dynamically when changes occur via administrative dashboards.

Cryptographic routines using the Android Keystore API are executing smoothly, generating secure keypairs inside hardware-backed storage wrappers. Enforced encryption checks add an overhead of approximately 8 ms per transaction packet, representing a negligible performance cost given the substantial security enhancements provided. The application containerization work using Docker has produced viable baseline images for testing, laying down a solid foundation for upcoming multi-node Kubernetes cluster configuration and horizontal auto scaling experiments.

Chapter 5

Conclusions and future work

Project Synthesis This project demonstrates a major step forward from manual, error-prone campus vehicle logging to a modern, automated system. By combining an Android client application written in Kotlin with an IoT gateway built on a Raspberry Pi, the system achieves fast, reliable vehicle permit checking at campus gates. The edge scanning system using NFC to read employee RFID tokens has proven robust, enabling quick identification and eliminating the delays common with paper-based checks.

The multi-tier system architecture—spanning local Bluetooth socket interfaces, Apache Kafka messaging brokers, PostgreSQL relational tracking tables, and containerized deployment packages via Docker—creates a highly scalable structure. The inclusion of Android Keystore private key cryptographic mechanisms ensures data privacy and protects personal information from unauthorized access. Overall, the current implementation provides a solid framework that successfully delivers on the core objectives outlined in the problem statement.

Identified Limitations and Future Roadmap Despite successful milestones in development, several limitations have been identified that form the basis of the future development roadmap. Currently, the Kafka clustering mechanism and the PostgreSQL operations are constrained to single-instance dev environments; transitioning these into a distributed cloud or cluster architecture is an immediate priority. Additionally, while the UI is highly functional, layout bugs on varied mobile displays need to be resolved before deployment.

The long-term objectives are heavily centred around infrastructure optimization and deployment scalability. Over the coming weeks, work will focus on finalizing multi-node Kubernetes configurations to enable automated failover management and horizontal scaling. The PostgreSQL integration will be expanded to support real-time data streaming queries, allowing administrative tools to monitor entry activity instantly. Finally, comprehensive field testing will be executed at campus gate facilities to measure the physical system’s durability, signal range, and real-world performance under diverse environmental conditions.

Bibliography

- [1] Android Developer Network. Android keystore system & nfc adapter apis. <https://developer.android.com/training/articles/keystore>, 2026. Official API Guides.
- [2] Kotlin Documentation Contributors. Kotlin docs. <https://kotlinlang.org/docs/home.html>, 2026. Online Resources.
- [3] MDN Web Docs Contributors. Xml documentation. <https://developer.mozilla.org/en-US/docs/Web/XML>, 2026. Online Reference.
- [4] Python Software Foundation. Socket — low-level networking interface. <https://docs.python.org/3/library/socket.html>, 2026. Python Standard Library Documentation.
- [5] Raspberry Pi Foundation. Raspberry pi hardware and software documentation. <https://www.raspberrypi.com/documentation/>, 2026. Online Tech Specs.